

CachePilot: 基于 Mooncake 的 KVCache 复用与调度评测系统

技术汇报 / 实机测试结果 / 演示说明

2026 年 Mooncake 赛题参赛作品

仓库: <https://github.com/imperson123/CachePilot>

真实 Store 实测
16 组

read_perf 全部 return_code=0

峰值带宽
1399.08 MiB/s

1MB, batch=4

正确性验证
0 failures

misses=0, verify_failures=0

汇报脉络

项目目标

- ▶ 面向 Mooncake KVCache 赛题，补充可复现实测与策略评估工具。
- ▶ 不侵入 Mooncake Store / Transfer Engine / RDMA 核心路径。
- ▶ 通过 CSV、日志、图表和 Dashboard 支撑提交文档与演示视频。

当前完成内容

模块	完成内容
Store	包装官方 store_kv_bench.py，完成真实 verify / read_perf 实测
Prefix	可控 workload 下分析 prefix hit 对 TTFT 的影响
Scheduler	对比 Random / Nearest / Reuse-aware / CachePilot
Dashboard	Streamlit 面板用于录屏与截图

赛题问题定位：KVCache 的可复现评测缺口

赛题关注点

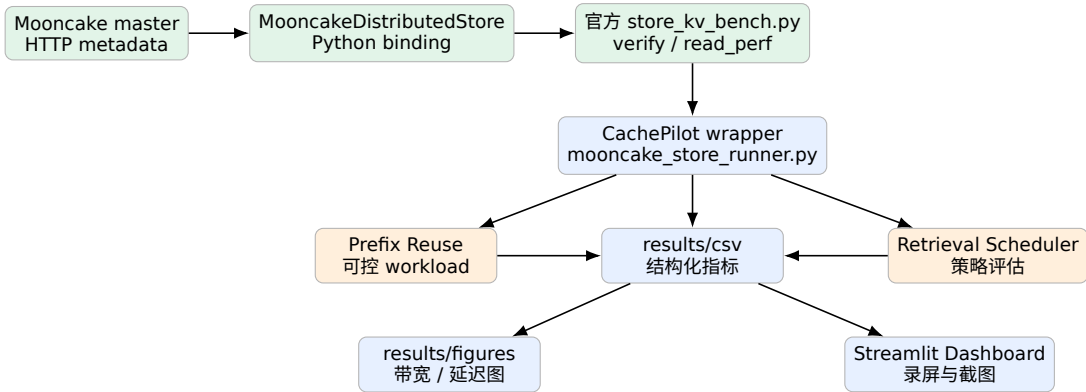
- ▶ KVCache 存储、复用、传输与调度。
- ▶ 长上下文、多轮对话、高并发服务下的 TTFT、吞吐与 P99。
- ▶ 与 Mooncake Store、HiCache、推理框架生态兼容。

CachePilot 切入点

- ▶ 先建立单机 TCP 下真实 Store I/O 基线。
- ▶ 再围绕 prefix reuse 和 retrieval scheduling 扩展策略指标。
- ▶ 输出结构化结果，降低复现实验和提交材料整理成本。

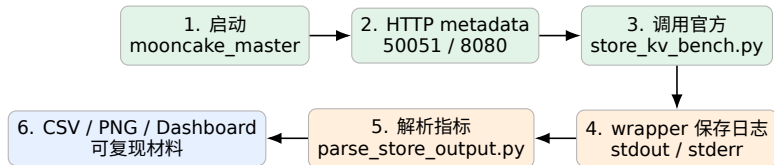
核心定位：**非侵入式 benchmark/evaluation suite**，不是替换 Mooncake 内核，而是补齐评测、记录与演示链路。

系统总体架构



绿色为真实 Mooncake Store 实机链路；橙色为可控 workload 策略评估；蓝色为结果输出与展示。

真实 Store Benchmark 数据采集流程



- ▶ `verify_write`: 写入后读回并验证 payload。
- ▶ `read_perf`: 覆盖 4KB、64KB、1MB、4MB 与 `batch=1/4/8/16`。
- ▶ 输出 req/s、kv/s、MiB/s 与 P50/P95/P99。
- ▶ 所有实测结果均保留原始日志与结构化 CSV。

实验环境与运行配置

配置项	实验设置
平台	AutoDL 容器实例
GPU	RTX 4090 24GB
CPU / 内存	16 核 / 120GB
系统	Ubuntu 22.04
Python /	Python 3.10 / CUDA
CUDA	11.8
Mooncake	mooncake-transfer-
包	engine-non-cuda
协议	TCP

```
export LD_PRELOAD=/usr/lib/x86_64-linux-gnu/libstdc
++.so.6
export MOONCAKE_ROOT=/root/autodl-tmp/
mooncake_real_run/Mooncake
export HOST_IP=$(hostname -I | awk '{print $1}')

mooncake_master --port=50051 \
--enable_http_metadata_server=true \
--http_metadata_server_port=8080

python benchmark/mooncake_store_runner.py \
--scenario read_perf --io-api plain \
--value-sizes 4096,65536,1048576,4194304 \
--batch-sizes 1,4,8,16 --runtime 5
```

功能验证: `verify_write` 成功

return_code**0**

真实 Store 链路可用

MiB/s**38.80**

4KB, batch=4

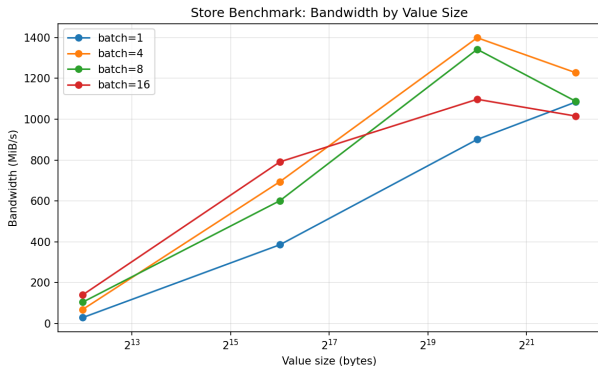
P99**0.781 ms**

misses=0, failures=0

value	batch	req/s	kv/s	MiB/s	p50	p95	p99	校验
4KB	4	2482.98	9931.94	38.80	0.260	0.718	0.781	misses=0, failures=0

- ▶ 该步骤确认 Python binding、master、metadata server 与 Store client 链路全部可用。
- ▶ 后续 `read_perf` 结果均基于同一套真实 Mooncake Store 链路。

真实 read_perf: 吞吐随对象大小变化

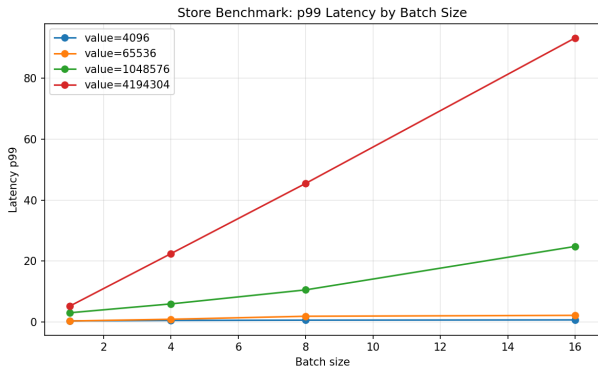


关键观察

- ▶ 小对象受请求开销影响明显。
- ▶ 对象增大后带宽快速上升。
- ▶ 1MB、batch=4 达到峰值 **1399.08 MiB/s**。
- ▶ 4MB 仍保持较高吞吐，但尾延迟上升。

value	batch	MiB/s
4KB	16	139.86
64KB	16	791.38
1MB	4	1399.08
4MB	4	1227.74

真实 read_perf: P99 尾延迟权衡



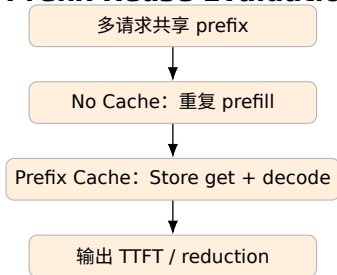
代表性结果

value	batch	p50	p99
4KB	1	0.106	0.380
4KB	16	0.352	0.703
1MB	4	2.574	5.956
4MB	4	12.251	22.456
4MB	16	52.517	93.267

- ▶ batch 和 value size 提升会增加单次传输负载。
- ▶ 4MB + batch=16 的 P99 显著上升，说明评测可揭示吞吐与尾延迟边界。

Prefix Reuse 与 Scheduler: 策略评估模块

Prefix Reuse Evaluation



Retrieval Scheduler Evaluation

- ▶ 构造多节点 KV block metadata: reuse、importance、fetch latency、size、replica。
- ▶ 策略对比: Random、Nearest、Reuse-aware、CachePilot。
- ▶ CachePilot score:

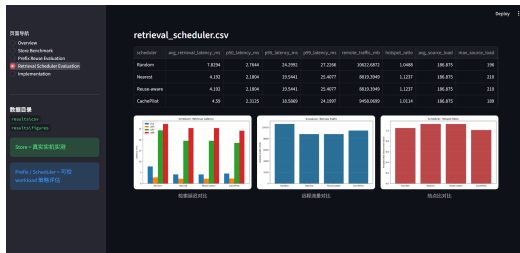
$$score = \alpha \cdot importance + \beta \cdot reuse - \gamma \cdot fetch\ latency$$

该部分是基于可控 workload 的离线策略评估，不宣称为完整分布式 LLM serving 端到端实测。

Dashboard: 面向演示视频与文档截图



Store Benchmark 页面: 真实带宽与 P99 趋势



Retrieval Scheduler 页面: 策略延迟、远程流量、热点比

- Dashboard 区分“Store 真实实机实测”和“Prefix / Scheduler 可控 workload 策略评估”。
- 适合比赛演示视频: 仓库说明 → 实测 CSV → Store 图表 → 策略页面 → 运行脚本。

工程问题与处理记录

问题	表现	处理方式
GLIBCXX_3.4.30 缺失	Python import Mooncake binding 报错	使用系统新版 libstdc++.so.6 进行 LD_PRELOAD
Python binding 与仓库脚本接口差异 4MB 高 batch 初次失败	get_alloc_func_addr 或 nof_replica_num 报错 默认 segment / buffer 偏小	本地实测环境对官方 benchmark 做兼容补丁 global segment 调至 512MB, local buffer 调至 256MB

这些问题说明：补充 benchmark 文档、依赖说明与自动化脚本，有助于社区复现 Mooncake Store 相关实验。

当前结论与后续扩展

当前结论

- ▶ Store Benchmark 已完成单机 TCP 实机实测。
- ▶ verify_write 成功, misses=0, verify failures=0。
- ▶ read_perf 16 组全部 return_code=0。
- ▶ 1MB、batch=4 取得 1399.08 MiB/s 峰值带宽。

后续扩展

- ▶ 扩展 RDMA / 多机 / zcopy / write_perf / mixed_rw。
- ▶ 接入 SGLang HiCache 或 vLLM 场景, 形成端到端 serving 评估。
- ▶ 用真实 Store latency 标定 Prefix Reuse 与 Scheduler 模型。
- ▶ 以 benchmarks/cachepilot/ 形式准备开源 PR。